

Chapter

06

지금까지 만든 프로그램들은 미리 준비한 데이터에 의존해 작동했습니다. 하지만 실제 서비스는 예측할 수 없는 사용자 입력을 처리해야 합니다. 예를 들어 사진 속 식재료로 레시피를 생성하는 애플리케이션에는 이미지를 분석하고 문장을 생성하는 AI 모델의 능력이 필요합니다.

이번 장에서는 AI 모델을 API로 연동해, 애플리케이션에 진짜 AI의 날개를 달아 보겠습니다. API의 기본 개념부터 AI 모델 선택과 통합까지, 클로드 코드로 AI 기반 웹 애플리케이션을 직접 구현하는 방법을 익혀 봅시다.

클로드 코드에 API 날개 달기

학습목표

- API의 개념을 이해하고 AI 모델 API를 선택할 수 있습니다.
- API 통합 플랫폼 또는 직접 연동 방식으로 AI 모델을 활용할 수 있습니다.
- 클라우드 코드에서 API 키를 관리하고 활용할 수 있습니다.
- 이미지 인식과 텍스트 생성 언어 API를 연동한 실제 애플리케이션을 만들 수 있습니다.

06-1

클로드 코드에서 API 설정하기

핵심 키워드

API

API 키

오픈라우터

.env 파일

API는 우리 프로그램과 외부 서비스를 연결하는 다리입니다. 이번 절에서는 다양한 AI 모델 API를 탐색하고 프로젝트에 맞는 모델을 선택하는 방법을 배워봅니다. API 통합 플랫폼의 활용법과 API 키를 안전하게 관리하는 방법, 그리고 클로드 코드를 이용해 이미지 인식과 텍스트 생성 기능을 실제로 구현하는 과정을 단계별로 실습해 보겠습니다. 프로젝트의 규모와 요구사항에 따라 적절한 API를 선택하고 효율적으로 연동하는 실무 기술을 익혀 봅시다.

시작하기 전에

레스토랑에서 요리를 맛보려면 주방장에게 직접 이야기하는 것이 아니라 웨이터에게 주문을 해야 합니다. 소프트웨어 개발에서도 마찬가지입니다. 복잡한 시스템을 직접 구축하는 것이 아니라, API에게 요청해서 필요한 결과만 받아올 수 있습니다. 클로드 코드를 사용하면 이런 API 연동 작업이 더욱 간편해집니다. 복잡한 연결 코드를 직접 작성할 필요 없이, 클로드 코드가 API 설정부터 오류 처리까지 모든 과정을 자동으로 처리해 줍니다. 이제 클로드 코드와 함께 API를 활용한 실제 애플리케이션을 만들어 봅시다.



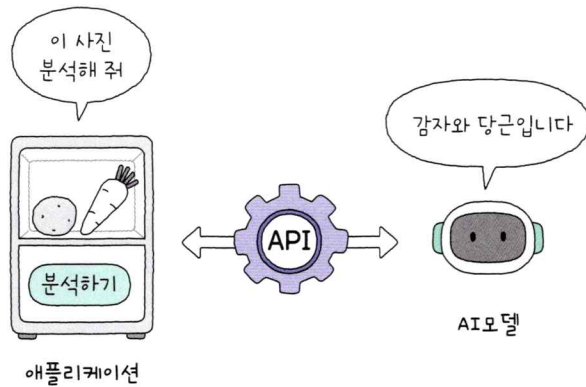
API의 개념과 활용 방법

지금까지 우리가 만든 웹 포트폴리오와 퀴즈 게임에는 공통점이 있습니다. 웹 포트폴리오에 소개될 내 정보나, 퀴즈 게임의 문제들은 ‘모두 미리 준비된 데이터’였다는 점입니다. 하지만 만약 다음과 같은 애플리케이션을 만들게 된다면 어떨까요?

사용자가 냉장고 사진을 업로드하면, 이미지 속 식재료를 자동으로 인식하고 그 재료를 활용한 레시피를 추천해 주는 프로그램

이 애플리케이션은 준비해 둔 정보만으로는 운영할 수 없습니다. 어떤 냉장고 사진이 업로드될지 알 수 없기 때문입니다. 따라서 이미지를 읽고 해석하는 비전 AI^{vision AI} 모델과 해석 결과를 바탕으로 레시피를 추천하는 생성 AI 모델이 필요합니다. 그러고 나면 다음 문제는 ‘이 모델들을 애플리케이션에 어떻게 통합할 것인가’입니다. 이때 유용한 것이 바로 API입니다.

API Application Programming Interface 는 프로그램과 AI 모델을 이어주는 다리입니다. 마치 전화로 전문가에게 도움을 요청하듯, 프로그램이 AI 모델에게 사진을 분석해 달라고 요청하면 AI가 이미지를 해석하고 결과를 돌려줍니다. 이후 애플리케이션은 해당 정보를 화면에 표시하거나, ‘이 재료로 레시피를 만들어 줘’와 같은 추가 요청을 AI 모델 API에 전달해 결과를 받아볼 수도 있습니다. 즉 웹 애플리케이션은 사용자가 사진을 업로드할 인터페이스만 제공하고, 이미지 인식과 레시피 생성 같은 핵심 기능은 AI 모델 API가 작업하게 하는 것입니다.



+ 여기서 잠깐

API를 사용하지 않고 AI 모델을 직접 만들 수도 있지 않나요?

앞서 만들었던 손글씨 인식 모델을 만들었던 것처럼, 원하는 AI 모델을 직접 개발할 수도 있습니다. 하지만 이미지 인식이나 텍스트 생성 같은 복잡한 AI를 만들려면 방대한 양의 학습 데이터와 고성능 컴퓨터 자원이 필요합니다. 특히 생성형 언어 모델을 만드는 것은 클로드나 챗GPT를 개발하는 것만큼이나 어렵고 많은 비용이 들어갑니다.

따라서 이미 구축된 AI 모델을 API를 통해 활용하는 방식이 훨씬 효율적입니다. 완성된 고성능 AI 모델을 바로 사용할 수 있기 때문에, 제한된 시간과 자원 안에서 프로젝트를 완성해야 하는 상황이라면 API는 가장 현실적인 선택입니다.

클로드 코드로 이러한 AI 모델 API를 사용해서, 냉장고 속 식재료 이미지를 분석한 뒤 레시피를 생성하는 웹 애플리케이션, ‘냉장고를 부탁해’를 효과적으로 만드는 방법을 알아봅시다.

AI 모델 선택하기

API는 제공 목적에 따라 종류가 다양합니다. 예를 들어 날씨를 예측하는 API, 뉴스를 수집하는 API, 지도를 제공하는 API 등이 있습니다. 많은 AI 개발사는 자사 모델을 API 형태로 제공해, 다른 프로그램에서도 활용할 수 있도록 해 줍니다.

클로드도 API를 제공합니다. 클로드의 API를 연결하면 사용자가 레시피를 요청할 때마다 클로드가 직접 대답하는 것처럼 결과를 받을 수 있습니다. 다만 클로드나 챗GPT처럼 유료로 운영되는 대형 AI 서비스의 경우, API를 사용할 때도 사용량에 따라 요금이 부과됩니다. 사용자가 급격히 증가할 경우 늘어나는 비용을 감당하기 어렵겠죠. 따라서 어떤 API를 선택할지는 신중히 결정해야 합니다.

다행히 학습 목적으로 충분한 성능을 제공하는 무료 API도 많이 있습니다. 지금처럼 기능을 익히거나 초기 버전을 만들 때는 이런 무료 API를 사용하는 것이 당연히 좋겠죠.

오픈라우터 OpenRouter는 여러 AI 모델 API를 한곳에 모아 제공하는 API 통합 플랫폼입니다. 유료 또는 무료 API를 한눈에 비교하고, 어떤 API가 인기 있는지도 살펴볼 수 있죠. 덕분에 원하는 API를 쉽게 찾아보고 내 프로젝트에 맞는 모델을 골라 사용할 수 있습니다. 이제 오픈라우터에 접속해 프로젝트에 적합한 API를 직접 선택해 보겠습니다.

+ 여기서 잠깐

오픈라우터를 사용하는 이유

AI 모델을 사용하는 방법은 크게 두 가지입니다.

첫 번째는 오픈라우터 같은 API 통합 플랫폼을 사용하는 방법입니다. 마치 푸드코트에서 여러 식당의 음식을 한 곳에서 주문하듯, 오픈라우터에서는 클로드 Claude, GPT, 라마 Llama 등 다양한 AI 모델을 한곳에서 제공합니다. API 키를 발급받기만 하면 모든 모델을 자유롭게 전환해 쓸 수 있고, 사용량도 한 화면에서 관리할 수 있어 매우 편리합니다.

두 번째는 각 AI 회사의 공식 API를 직접 사용하는 방법입니다. 예를 들어 오픈AI에서 GPT API를, 앤트로픽에서 클로드 API를 직접 연동하는 것이죠. 이 경우 회사마다 회원가입을 하고 별도의 API 키를 발급받아야 하므로 관리가 다소 번거롭지만, 최신 기능을 가장 빠르게 이용할 수 있습니다.

초보자라면 오픈라우터로 시작하는 것을 추천합니다. 여러 모델을 쉽게 비교해 볼 수 있고, 무료 모델로 체험할 수 있는 모델도 많기 때문입니다. 다만 무료 사용의 경우, 이 글을 쓰는 2025년 11월 기준 분당 20회, 하루 50회의 요청만 가능합니다. 따라서 특정 모델을 꾸준히 사용하게 된다면, 그때 회사의 공식 API를 직접 연동해 보는 것도 좋은 선택이 될 것입니다.

API 키 발급하기

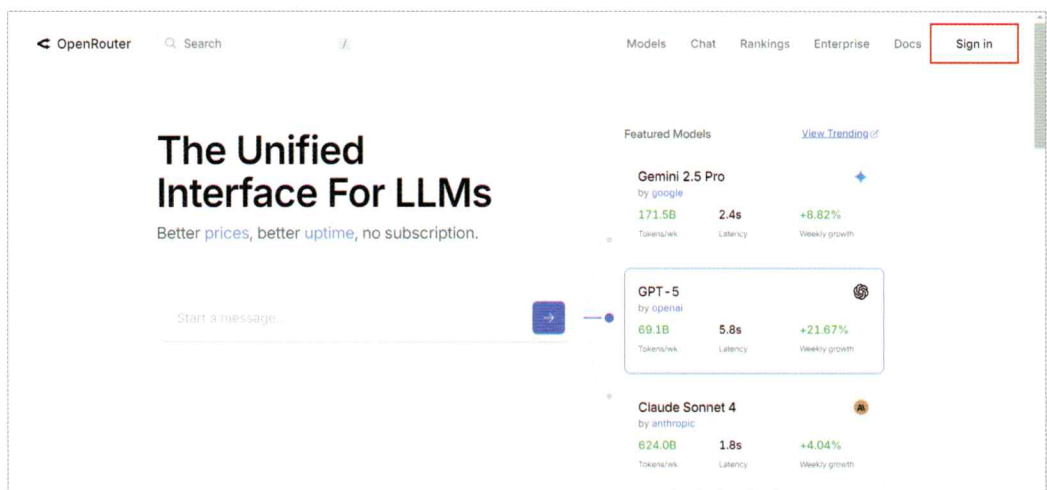
API 키 API key는 말 그대로 API를 사용할 때 사용자를 인증하는 비밀 열쇠입니다. API를 안전하고 공정하게 사용하기 위한 인증과 관리의 핵심 도구죠. API를 제공하는 서버는 이 키를 통해 누가 요청을 보냈는지, 그리고 그 요청이 허가된 사용자에게서 온 것인지를 확인합니다.

API 키가 없다면 누구나 서버에 접근할 수 있게 되어 데이터 유출의 위험이 커질 수 있습니다. 또한 서버는 API 키를 이용해 사용자의 호출 빈도나 사용량을 기록하여 과도한 요청을 제한하거나 요금 부과를 관리하는 데도 활용합니다.

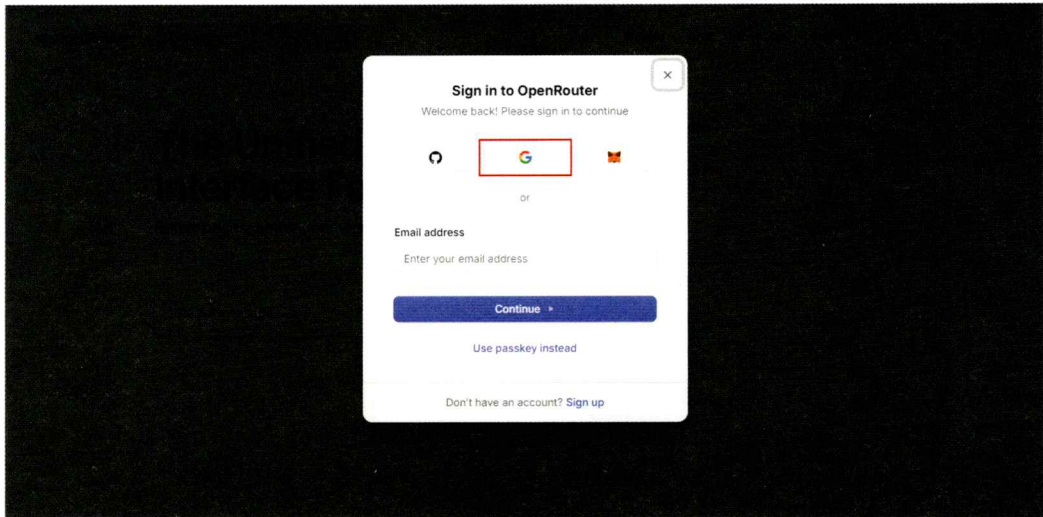
오픈라우터에서 API를 사용할 때도 API 키가 필요합니다. 지금부터 API 키를 발급받아 봅시다. ‘VibeCoding’ 폴더에 ‘Study-04’ 폴더를 새로 생성해 두세요.

01 웹 브라우저를 열고 오픈라우터에 접속한 뒤, 홈페이지 상단의 [Sign in] 버튼을 클릭해 회원가입을 시작합니다.

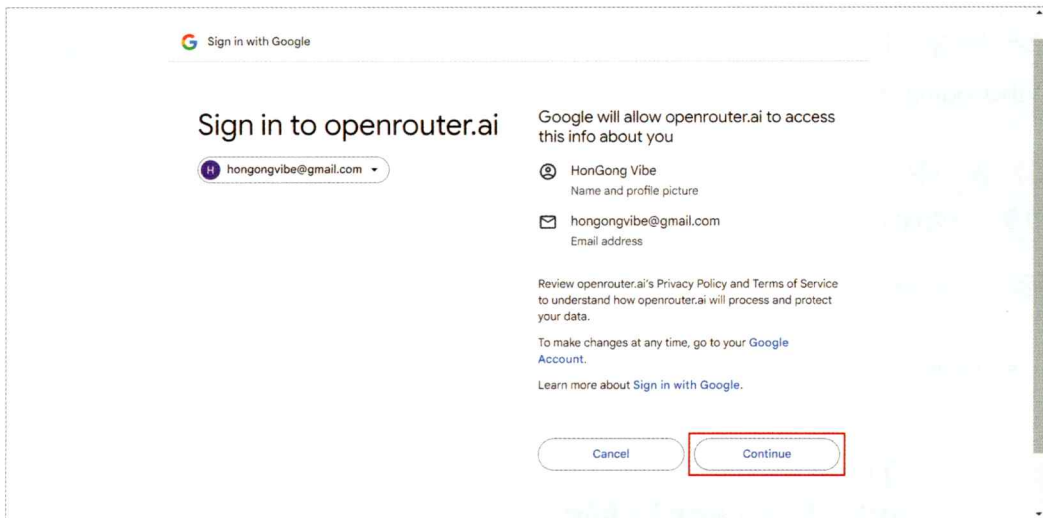
[URL](https://openrouter.ai) openrouter.ai



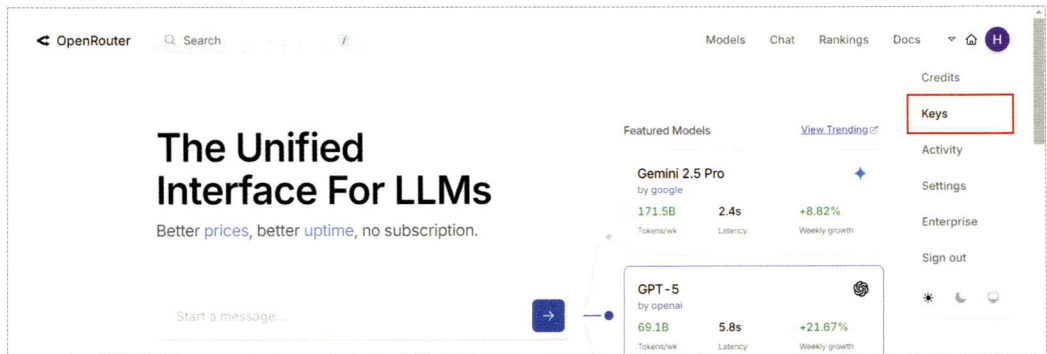
02 이 책에서는 구글 계정 연동을 통해 가입해 보겠습니다. 가운데 구글 로고를 클릭하세요.



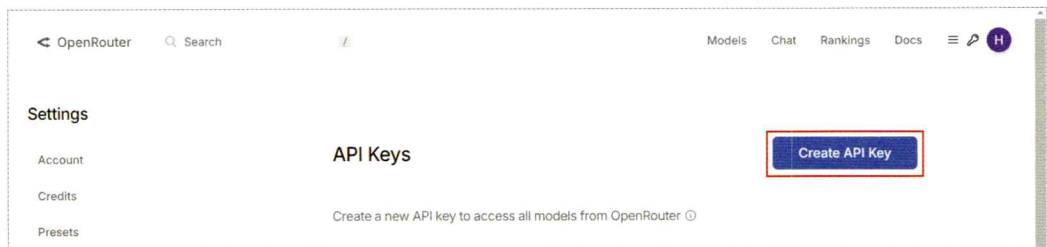
03 구글 아이디를 입력한 뒤 Continue 버튼을 클릭해 가입을 진행합니다. 액세스 약관에 동의하고 Continue를 눌러 회원 가입을 마칩니다.



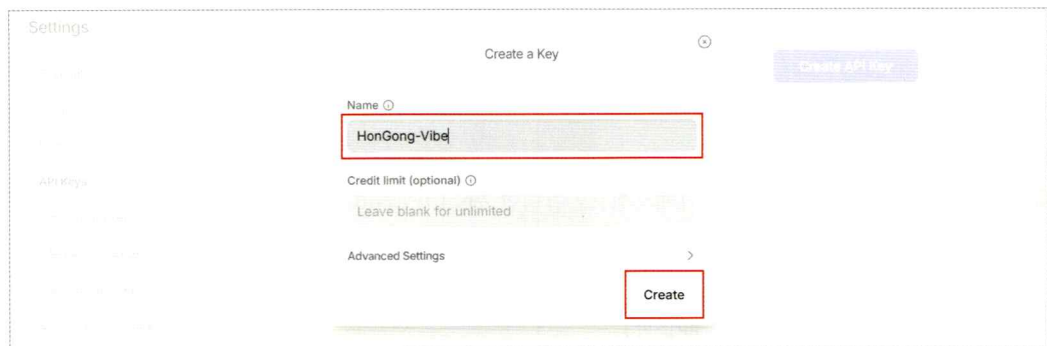
04 오픈라우터의 초기 화면으로 돌아가 API 키를 등록하기 위해 우측 상단의 프로필 아이콘을 클릭한 뒤 [Keys] 메뉴를 선택합니다.



05 [Create API Key] 버튼을 클릭합니다.

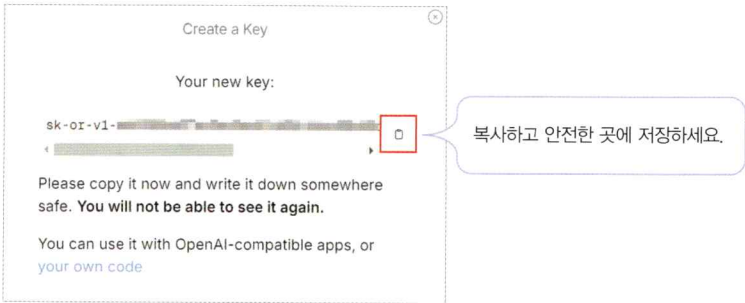


06 'Name' 입력란에 'HonGong-Vibe'라고 입력하여 이름을 붙여 주겠습니다. [Create] 버튼을 클릭하면 API 키를 발급받을 수 있습니다.



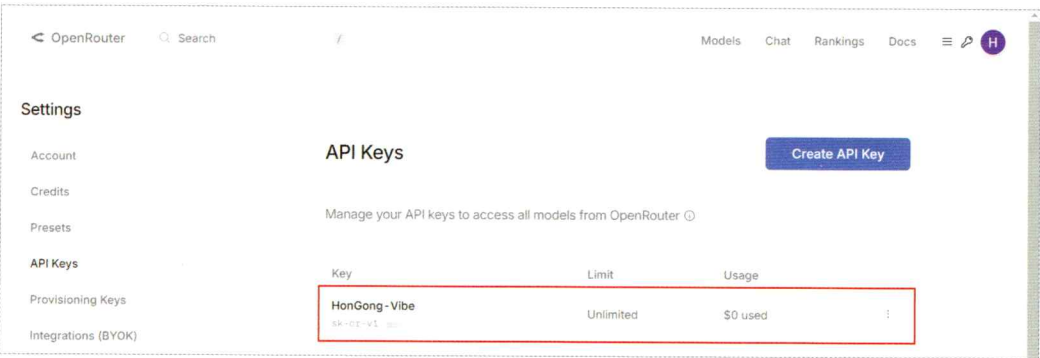
note 나중에 여러 프로젝트를 진행하게 되면 필요에 따라 추가 키를 만들 수도 있습니다.

07 실제 API 키가 화면에 표시됩니다. 이 API 키는 보안상의 이유로 단 한 번만 확인할 수 있습니다. 페이지를 벗어나거나 새로고침하면 다시 볼 수 없으므로, 반드시 안전한 곳에 복사해서 저장해야 합니다. 화면의 [📄(복사)] 아이콘을 클릭한 뒤, 메모장이나 텍스트 파일을 열어 붙여넣기하고 저장합니다. 이 키는 나중에 충전 후 유료 API를 사용할 때 필요한 열쇠이므로 다른 사람에게 노출되지 않도록 주의 깊게 관리해야 합니다.



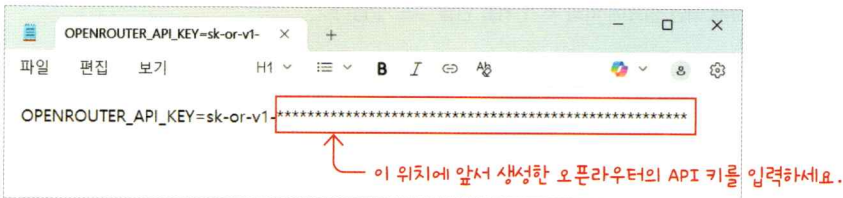
note API 키를 저장한 텍스트 파일은 절대 공용 클라우드 저장소에 업로드하면 안 됩니다. 자동 수집 봇에 의해 탈취되어 악용될 수 있습니다. 공용 폴더 대신 개인 로컬 컴퓨터에만 보관하세요.

08 API 키가 안전하게 생성되었습니다.

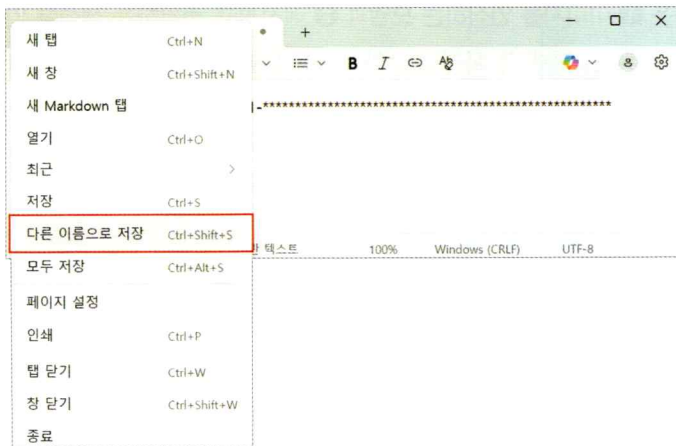


09 윈도우 기본 텍스트 편집기인 [메모장]을 실행한 뒤, 'OPENROUTER_API_KEY='라고 입력하고 '=' 기호 뒤에 앞서 발급받은 오픈라우터의 API 키를 붙여넣으세요.

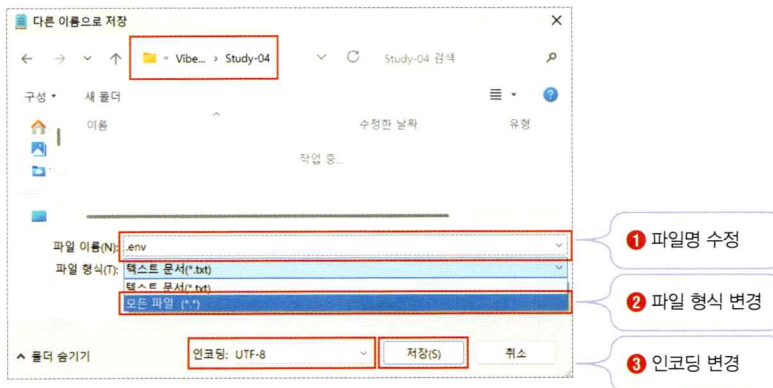
▶ OPENROUTER_API_KEY= < 오픈라우터 API 키 >



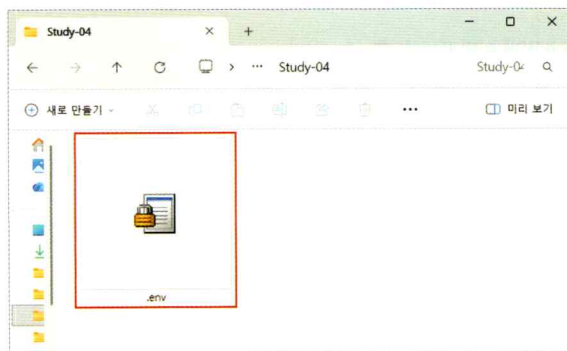
10 [파일] - [다른 이름으로 저장] 메뉴를 클릭하세요.



11 'Study-04' 폴더로 이동한 뒤, [파일 이름]에 '.env' 라고 입력합니다. 그리고 나서 [파일 형식]을 반드시 [모든 파일 (*.*)]로 변경해야 합니다. 하단의 [인코딩]은 [UTF-8]을 선택하고 [저장] 버튼을 클릭합니다.



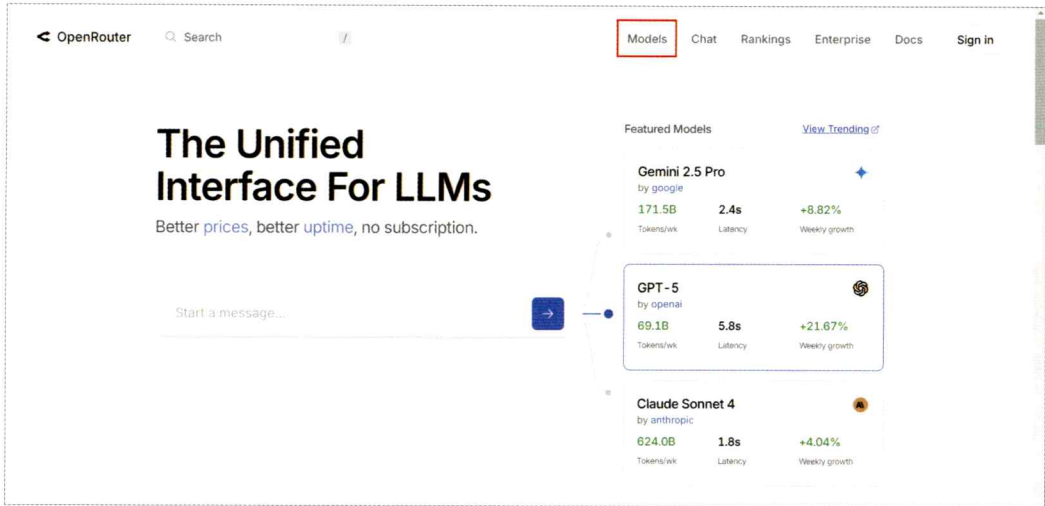
12 'Study-04' 폴더에 오픈라우터 API 키가 저장된 '.env' 파일이 생성된 것을 확인합니다.



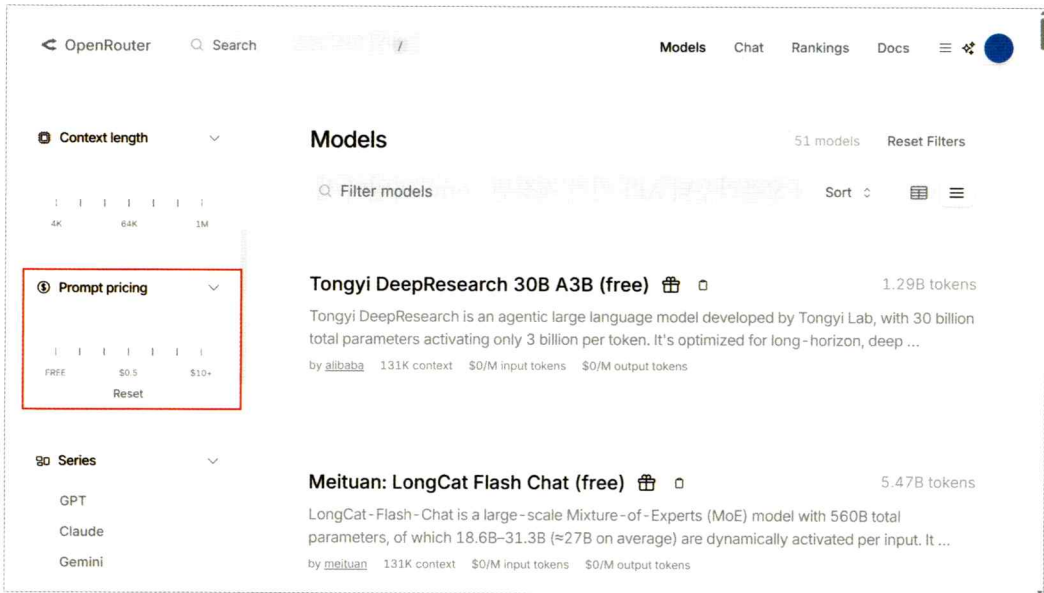
무료 AI 모델 탐색

‘냉장고를 부탁해’ 앱에 필요한 모델은 ❶ 이미지를 인식하는 모델과 ❷ 레시피를 추천하는 **생성형 언어 AI 모델**입니다. 오픈라우터가 제공하는 AI 모델 중 목적에 부합하면서 무료로 제공되는 것을 필터링하고, 적절한 성능을 보여주는 것을 선택해 봅시다.

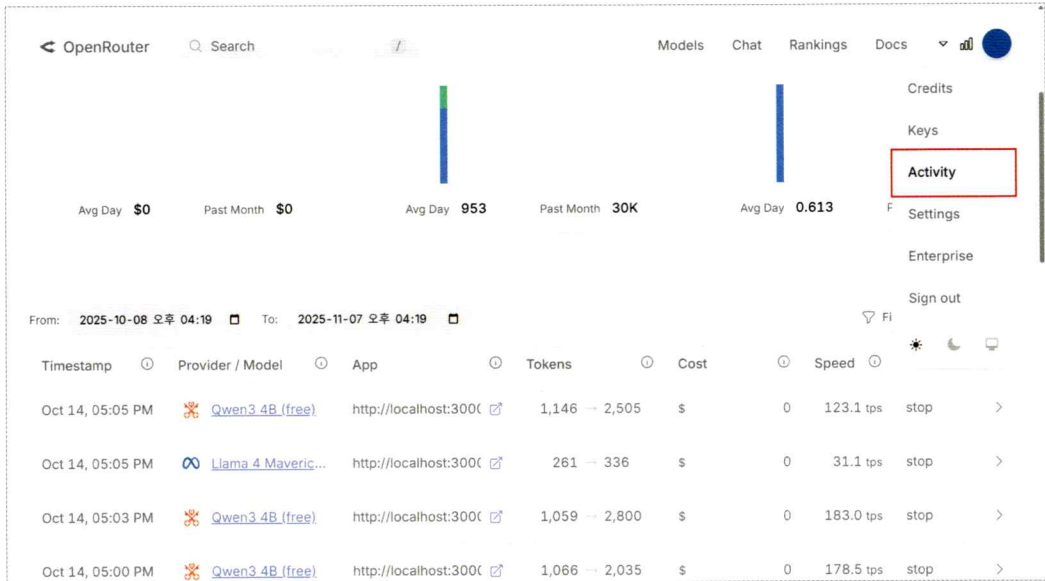
01 오픈라우터 초기 화면의 상단에서 [Models] 메뉴를 클릭하세요.



02 무료 API만 필터링하기 위해 좌측 사이드바에서 ‘Prompt pricing’ 필터를 찾고 슬라이더의 핸들을 드래그해 ‘FREE’에 맞추도록 하겠습니다.



오픈라우터에서 'free'로 표시된 모델들은 사용 자체는 무료이지만, 분당 요청 횟수(rate limit) 및 하루 요청 횟수 제한이 있습니다. 학습 목적으로는 이러한 제한이 있어도 충분히 활용 가능하며, 사용 기록은 오픈라우터의 [Activity] 메뉴에서 확인할 수 있습니다. 물론 나중에 실제 서비스를 운영하게 되어 많은 사용자가 동시에 접속한다면 유료 모델이나 크레딧 충전을 고려해야 합니다.

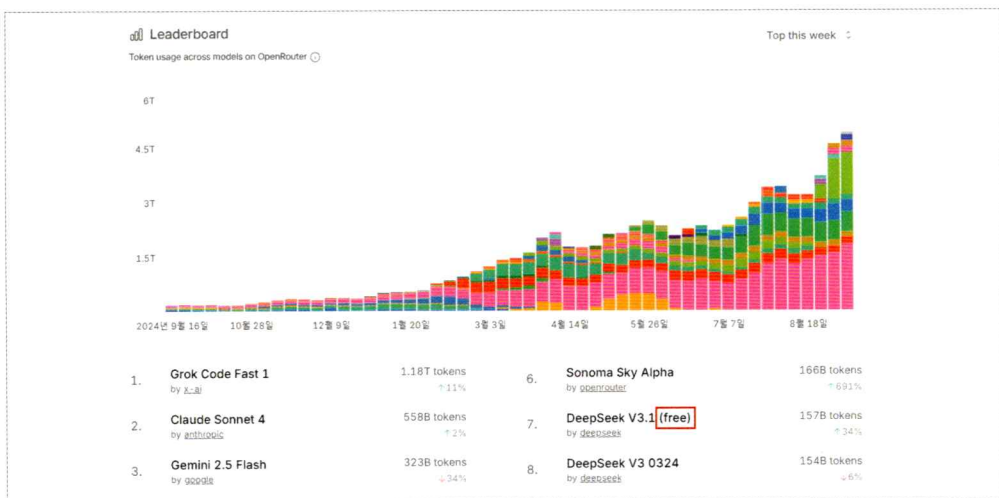


+ 여기서 잠깐

AI 모델의 순위를 확인하는 방법

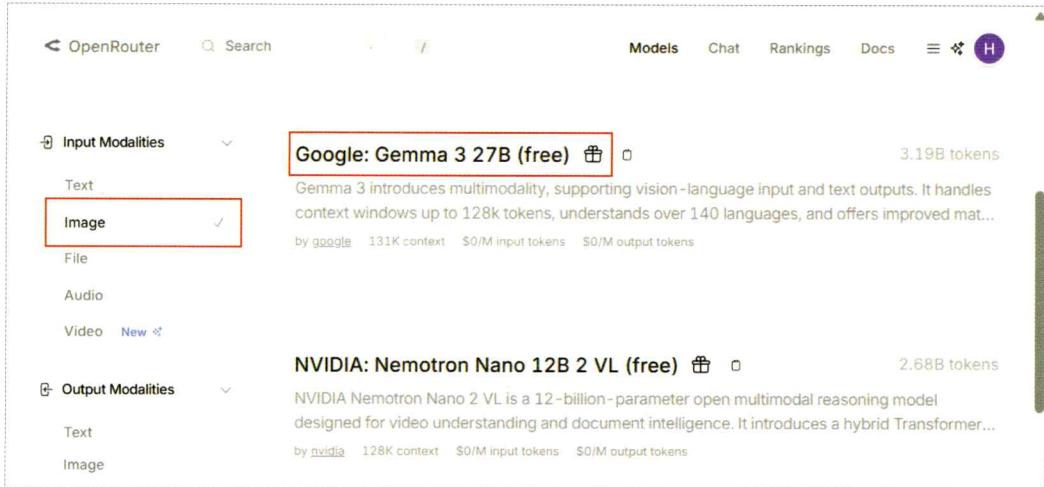
오픈라우터 상단의 [Ranking] 메뉴를 클릭하면 여러 모델들의 인기 순위를 한눈에 볼 수 있습니다. 가장 많이 사용되는 모델들이 사용량 그래프와 함께 표시되며 대부분 유료 API지만 'free' 표시가 있는 무료 모델도 순위에 포함되어 있습니다.

이 책에서는 무료이면서도 많은 사람들의 선택을 받아 랭킹에 오른, 성능이 검증된 모델을 선택해 사용하겠습니다.



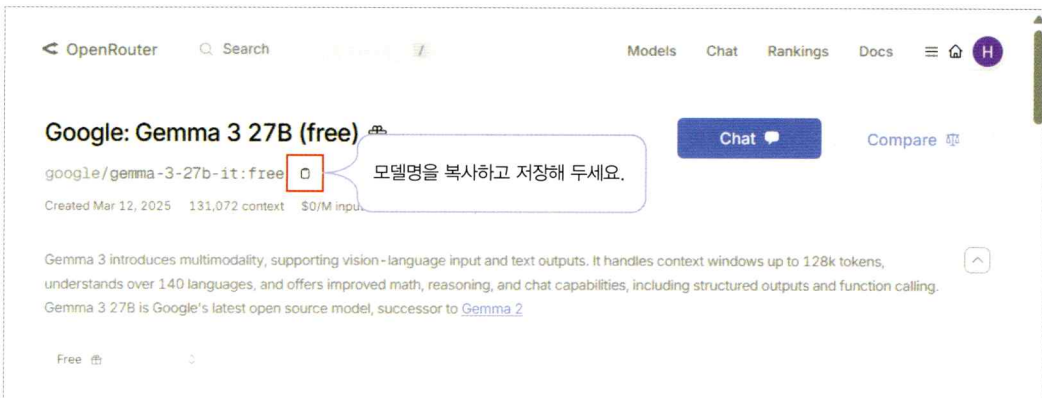
이미지 인식 AI 모델 탐색

01 좌측 사이드바의 'Input Modalities' 필터에서 [Image]를 선택합니다. 그러면 이미지의 입력과 처리와 관련된 모델이 필터링됩니다. 이 중 원하는 모델을 입력하세요. 여기서는 [Google: Gemma 3 27B(free)]를 클릭해 보겠습니다.



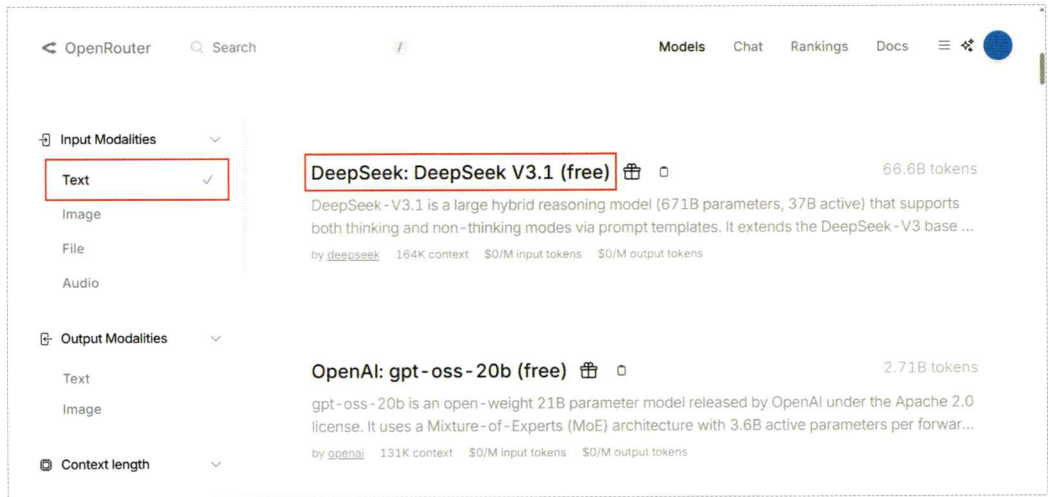
note 모델 검색 결과는 수시로 변동됩니다. 각자 원하는 모델을 선택해 진행하세요.

02 [📄(복사)] 아이콘을 클릭해 모델명을 복사하고 텍스트 파일에 저장합니다.



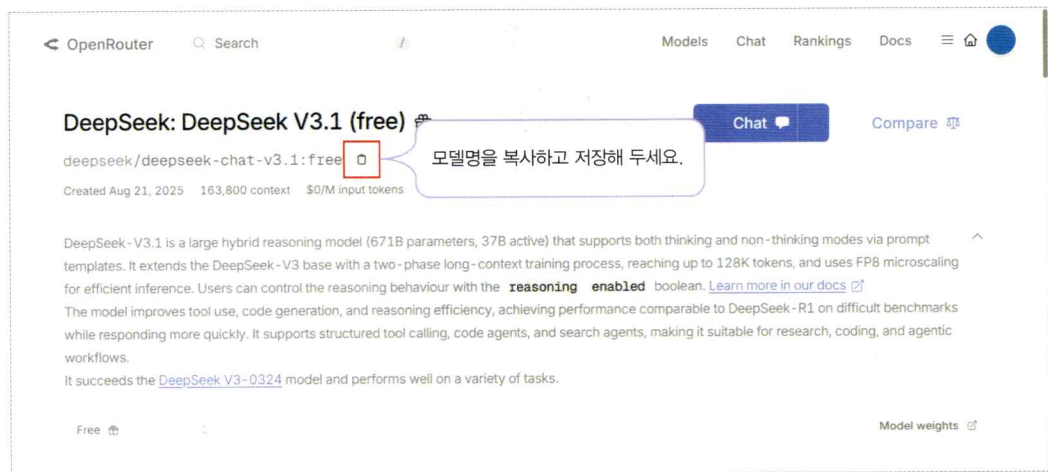
텍스트 생성 AI 모델 탐색

01 'Input Modalities' 필터에서 [Text]를 선택합니다. 여기서는 필터링된 모델 중 [DeepSeek: DeepSeek V3.1 (free)]를 선택해 보겠습니다.



note 책에서 제시한 모델이 보이지 않으면 원하는 모델을 선택해도 됩니다. 참고로 앞서 이미지 인식 모델로 선택한 [Google: Gemma 3 27B(free)]도 텍스트 생성을 지원하므로 해당 모델을 그대로 사용해도 좋습니다.

02 [📄(복사)] 아이콘을 클릭해 모델명을 복사하고 텍스트 파일에 저장합니다.

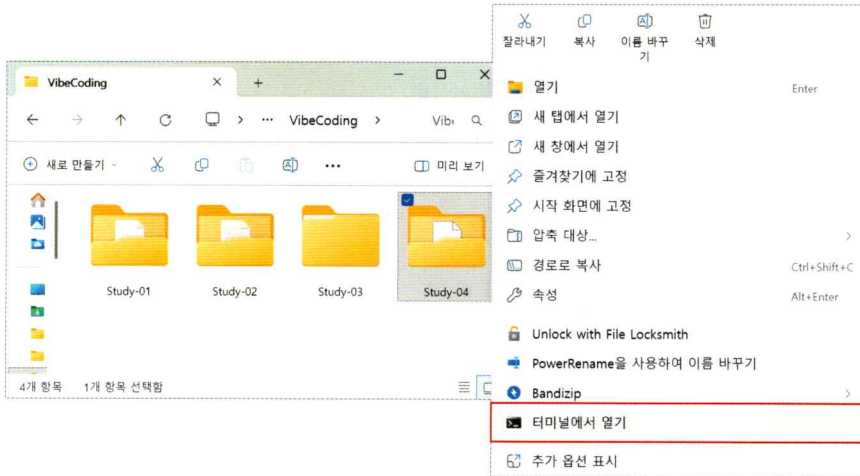


지금까지 오픈라우터의 API 키, 그리고 사용할 AI 모델들의 이름을 확인했습니다. 이제 API를 사용할 준비가 되었습니다.

AI 모델 연결하고 테스트하기

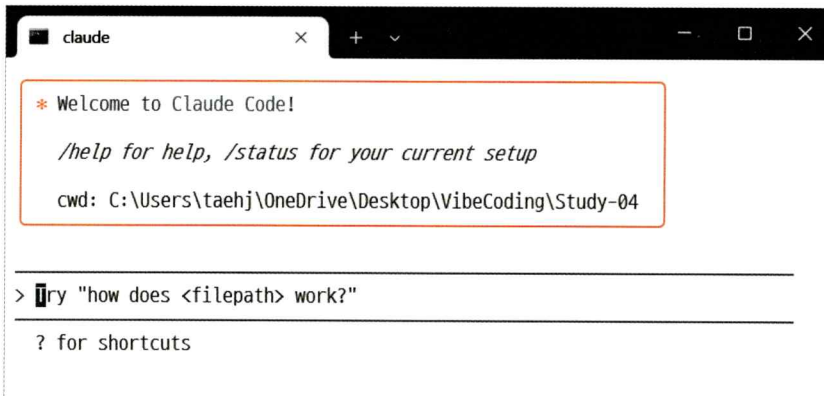
클로드 코드에 API를 연동해 레시피 추천 애플리케이션, ‘냉장고를 부탁해’를 만들 차례입니다.

데스크톱의 파일 탐색기를 열어 ‘VibeCoding’ 폴더의 ‘Study-04’ 폴더를 선택하고 마우스 오른쪽 버튼을 클릭한 뒤, [터미널에서 열기] 메뉴를 클릭해 터미널을 실행합니다.

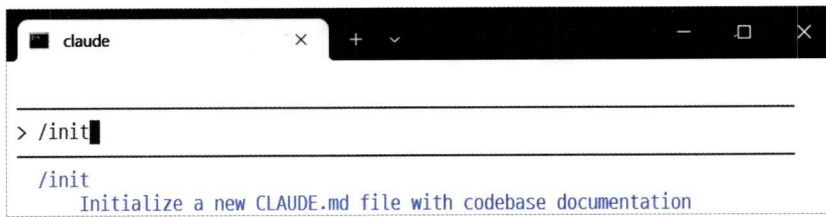


API 키 등록하기

01 터미널에 ‘claude’를 입력해 클로드 코드를 실행합니다.



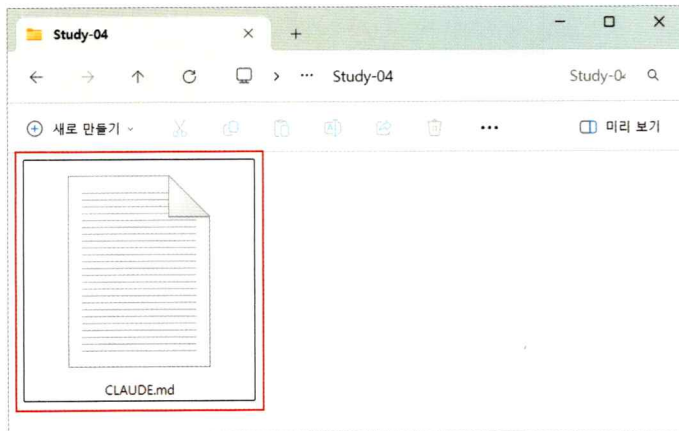
02 프로젝트 초기 설정을 위해 '/init' 명령어를 입력합니다.



```
> /init

/init
Initialize a new CLAUDE.md file with codebase documentation
```

03 'Study-04' 폴더에 'CLAUDE.md' 파일이 생긴 것을 확인합니다.



04 이제 오픈라우터에서 받은 API 키를 저장한 '.env' 파일을 클로드 코드에서 사용할 수 있게 해 보겠습니다. 클로드 코드에 다음과 같이 프롬프트를 입력하세요.

프롬프트

OpenRouter에서 받은 API 키를 .env 파일로 저장했어. 이 키를 안전하게 사용할 수 있도록 준비해 줘.

```
> OpenRouter에서 받은 API키를 .env 파일로 저장했어. 이 키를 안전하게 사용할 수 있도록 준비해 줘.█
```

Thinking off (tab to toggle)

note API 키를 프롬프트에 직접 입력해 작업하는 것은 보안상 문제로 권하지 않습니다.

05 API 키가 안전하게 설정되었습니다.

API Key Setup

- 완료했습니다! OpenRouter API 키를 안전하게 사용할 수 있도록 다음과 같이 설정했습니다:

생성된 파일들:

1. `.gitignore` - `.env` 파일이 Git에 커밋되지 않도록 보호합니다
2. `.env.example` - 다른 개발자들을 위한 환경 변수 템플릿
3. `config.js` - 환경 변수를 안전하게 로드하고 검증하는 유틸리티
4. `example.js` - OpenRouter API를 사용하는 예제 코드
5. `package.json` - Node.js 프로젝트 설정 파일

사용 방법:

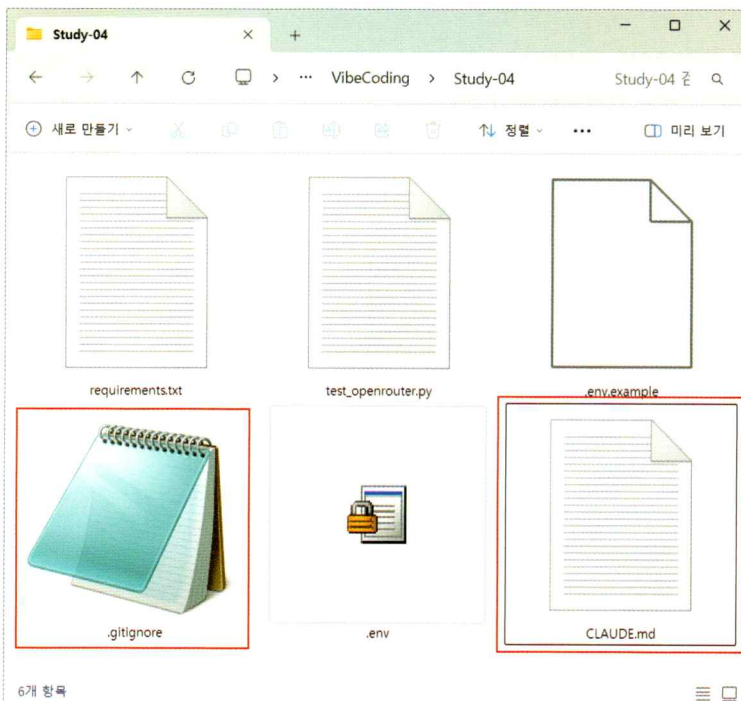
```
// 다른 파일에서 API 키 사용하기
const config = require('./config');

const apiKey = config.openRouter.apiKey;
const baseUrl = config.openRouter.baseUrl;
```

보안 기능:

- `.env` 파일은 Git 버전 관리에서 제외됨

06 'Study-04' 폴더를 확인하면 다음과 같이 '.gitignore' 파일과 그 밖에 보조적인 설정 파일, 프로젝트 관리를 담당하는 파일들이 새로 생성된 것을 알 수 있습니다.



생성된 파일은 터미널상의 설명을 통해 확인할 수 있습니다. 이 중 중요한 것은 다음 두 개의 파일입니다.

- **.env**: API 키가 저장된 환경 변수 파일, API를 호출할 때마다 참조됨
- **.gitignore**: '.env' 파일이 자동으로 깃허브에 업로드되지 않도록 특정 파일을 제외하여 보호하는 설정 파일

+ 여기서 잠깐 'env'와 '.gitignore' 파일의 이해

'env' 파일은 프로그램의 설정값을 저장하는 특별한 파일입니다. API 키처럼 민감한 정보를 프로그램 코드와 분리하여 관리하는 표준 방식입니다. 클라우드 코드는 API 키 설정 시 자동으로 '.env'와 '.gitignore' 파일을 생성함으로써 이러한 실무 관행을 자연스럽게 따르고 있습니다.

코드에 API 키를 직접 넣으면 깃허브에 업로드할 때 누구나 볼 수 있게 됩니다. 하지만 '.env' 파일을 '.gitignore'에 등록해 두면 자동으로 제외되므로 안전합니다. 또한 개발 중에는 무료 API를 쓰다가 실제 서비스에서는 유료 API로 전환해야 할 때도 코드 수정 없이 '.env' 파일만 바꾸면 됩니다. 팀 프로젝트에서도 유용합니다. 각자 다른 API 키를 사용해야 할 때, 코드는 공유하되 각자의 '.env' 파일만 따로 설정하면 충돌 없이 개발할 수 있습니다.

API 연동 테스트하기

01 이미지 인식은 구글의 Gemma 모델을, 생성 언어 모델은 DeepSeek 모델을 사용하겠습니다. 저장해 둔 각 모델의 이름을 확인해서 프롬프트에 다음과 같이 입력합니다.

프롬프트

이제 준비된 API가 실제로 작동하는지 테스트해 줘.

이미지 인식은 google/gemma-3-27b-it:free 모델을 이용하고,

텍스트 인식은 deepseek/deepseek-chat-v3.1:free 모델을 이용할 거야.

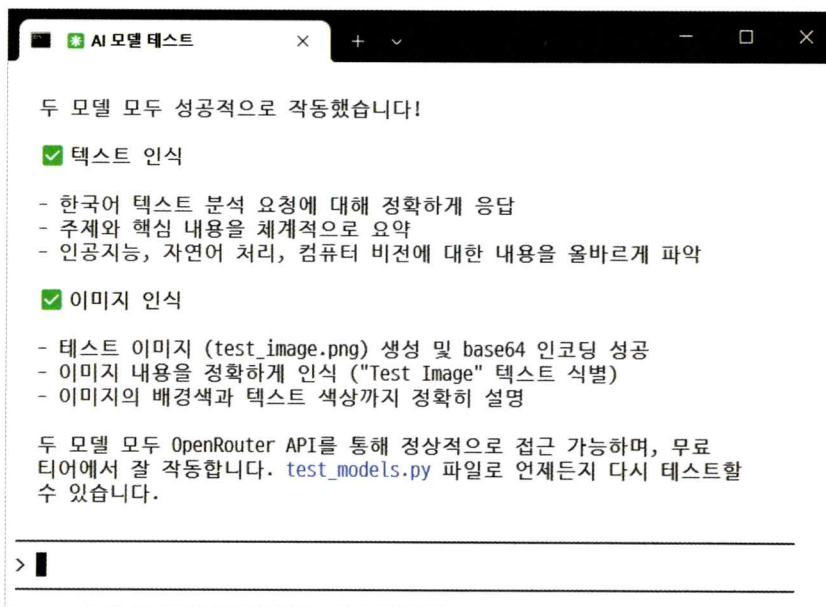
API를 통한 텍스트와 이미지 인식을 각각 테스트해서 실행 결과를 알려줘.

> 이제 준비된 API가 실제로 작동하는지 테스트해 줘.
이미지 인식은 google/gemma-3-27b-it:free 모델을 이용하고,
텍스트 인식은 deepseek/deepseek-chat-v3.1:free 모델을
이용할 거야.
API를 통한 텍스트와 이미지 인식을 각각 테스트해서 실행
결과를 알려줘.■

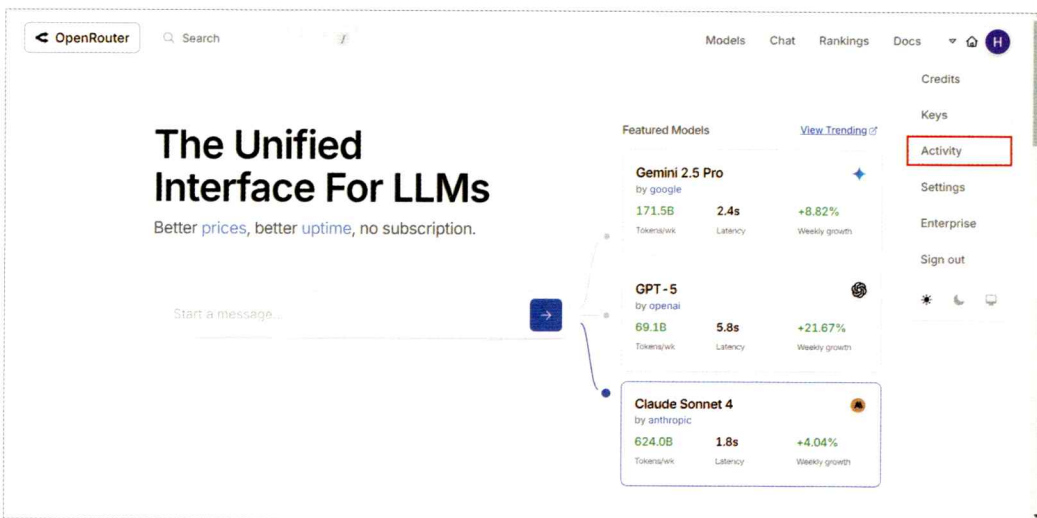
▶▶ accept edits on (shift+tab ctrl-g to edit prompt in
to cycle) start

note 오픈라우터에서 API를 적용할 때는 반드시 모델명을 정확히 입력해야 합니다.

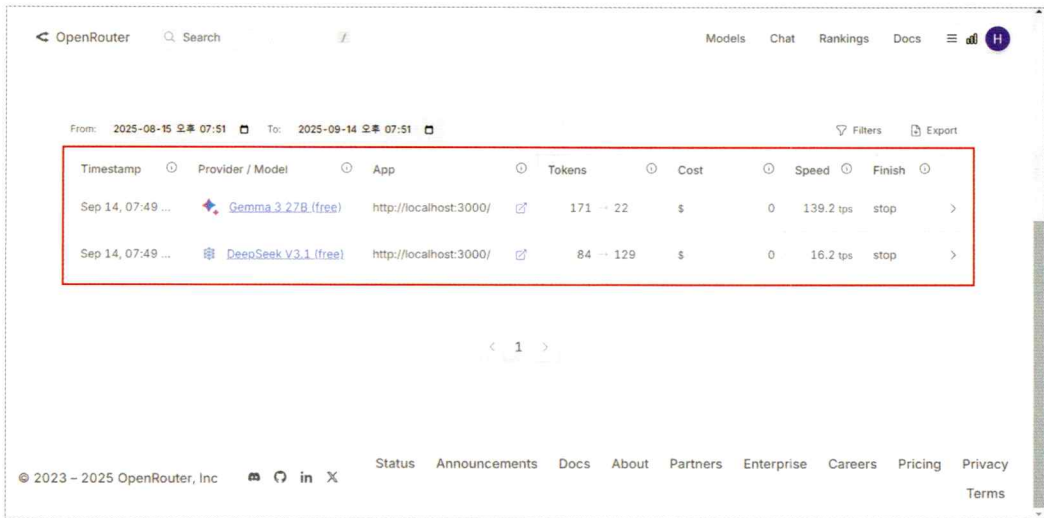
02 API 테스트에 성공했습니다.



03 실제로 API 키가 활용되었는지는 오픈라우터에서 확인할 수 있습니다. 오픈라우터 홈페이지에 로그인해서 프로필 아이콘을 클릭한 후, [Activity] 메뉴를 선택합니다.



04 'Timestamp' 열에 실제로 Gemma와 DeepSeek 모델을 사용한 기록을 확인할 수 있습니다.



Timestamp	Provider / Model	App	Tokens	Cost	Speed	Finish
Sep 14, 07:49 ...	 Gemma 3.27B (free)	http://localhost:3000/	171 → 22	\$	0 139.2 tps	stop >
Sep 14, 07:49 ...	 DeepSeek V3.1 (free)	http://localhost:3000/	84 → 129	\$	0 16.2 tps	stop >

지금까지 클로드 코드에 API를 연동했습니다. 이제 외부의 AI 모델을 사용해 '냉장고를 부탁해' 애플리케이션을 제작해 봅시다.

마무리

▶ 5가지 키워드로 정리하는 핵심 포인트

- API는 외부 AI 서비스와 프로그램을 연결하는 다리로, 간단한 요청만으로 복잡한 AI 기능을 활용할 수 있게 해 주는 통신 규칙입니다.
- 오픈라우터와 같은 API 통합 플랫폼에서는 여러 AI 모델을 한 곳에서 비교하고 선택할 수 있습니다.
- API 서비스에 접근하기 위해서는 API 키를 발급받아 사용자 인증을 해야 합니다.
- API를 통해 다양한 AI 모델을 애플리케이션에 통합하려면 AI 모델을 연동해야 합니다. 이때 클로드 코드의 작업 폴더에 다음과 같은 설정 파일을 생성하게 하세요.
 - .env: API 키를 저장한 파일로, API를 호출할 때마다 참조됨
 - .gitignore: API 키가 유출되는 것을 방지하기 위해 깃허브에 특정 파일이 업로드되지 않게 보호하는 설정 파일

▶ 확인 문제

1. 오픈라우터에서 API를 사용하기 위해 필요한 것을 모두 고르세요.

- | | |
|---------|-----------------|
| ① API 키 | ② 모델명 |
| ③ 자체 서버 | ④ .gitignore 파일 |

2. 클로드 코드에서 API 키를 안전하게 관리하기 위한 방법이 아닌 것을 고르세요.

- | | |
|---------------------|--------------------------|
| ① .env 파일에 API 키 저장 | ② .gitignore 파일에 .env 추가 |
| ③ 코드에 직접 API 키 하드코딩 | ④ 프로젝트 폴더 내에서만 키 참조 |

3. 오픈라우터의 Activity 메뉴에서 확인할 수 있는 정보가 아닌 것은?

- | | |
|-------------|-------------|
| ① 사용한 모델명 | ② 사용 시간 |
| ③ 호출한 코드 내용 | ④ API 호출 기록 |

06-2

클로드 코드와 API로 만드는 ‘냉장고를 부탁해’

핵심 키워드

다중 AI 모델 연동

웹 프레임워크

API를 활용해 일상의 문제를 해결하는 실용적인 애플리케이션을 직접 만들어 봅시다. 애플리케이션 구현을 위해 PRD를 작성하고, 제작 과정을 체계화하는 과정에서 실무에서 사용하는 효율적인 개발 방법론을 적용해 볼 것입니다. 체계적인 애플리케이션 개발 방법을 따라 단계별로 기능을 추가하고 테스트하며 실전 개발 프로세스를 경험할 수 있습니다.

시작하기 전에

누구나 한 번쯤 냉장고를 열고 ‘오늘 뭐 먹지?’하는 고민을 해 본 적이 있을 겁니다. 클로드 코드를 사용하면 이런 일상의 문제를 해결하는 실용적인 앱을 직접 만들 수 있습니다. 이번 절에서 제작할 애플리케이션은 ‘냉장고를 부탁해’입니다. 애플리케이션에 냉장고 내부를 찍은 사진을 업로드하면 식재료를 인식하고, 가지고 있는 재료에 따라 적절한 레시피를 추천합니다.



애플리케이션 구현을 위한 PRD 작성하기

이제 ‘냉장고를 부탁해’ 애플리케이션 제작을 위한 프롬프트를 만들 차례입니다. 프롬프트는 작업의 방향과 품질을 결정하는 핵심 요소이므로, 세부 내용을 점검하고, 실행 단계를 구체화하여 필요한 요구사항을 명확히 하는 과정이 필수적입니다. 이번에도 PRD를 만들어 진행해 보겠습니다.

이전 장까지는 클로드 웹에서 프롬프트를 만들어 확인하고 클로드 코드에 이를 복사해 넣는 식으로 진행했었죠. 이번에는 클로드 코드에서 직접 진행해 보겠습니다. ‘.md’ 파일을 활용하면 이전에 했던 것처럼 여러 단계의 세부 실행 프롬프트를 PRD 형식으로 구조화하여 작성할 수 있습니다.

단계별 계획 수립하기

‘냉장고를 부탁해’ 애플리케이션에는 두 가지 AI 모델이 필요합니다. 하나는 냉장고 사진 속 식재료를 분석하는 이미지 인식 모델이고, 다른 하나는 식재료를 활용해 레시피를 제안하는 텍스트 생성 모델입니다. 이처럼 **다중 AI 모델 연동**이 필요한 복잡한 애플리케이션을 개발할 때는 전체 프로젝트를 작은 단위로 나누어 점진적으로 개발하는 방법이 효율적입니다.

‘냉장고를 부탁해’ 애플리케이션은 핵심적인 기능을 먼저 구현한 뒤, 그것이 잘 작동하는지 확인한 다음, 세부적인 기능을 추가하는 방식으로, 단계별로 구현할 것입니다. 이렇게 하면 문제가 발생하더라도 각 단계에서 바로 수정할 수 있고, 점진적으로 완성도를 높일 수 있습니다.

웹 애플리케이션의 제작 과정을 3단계로 나누고 PRD 제작을 위해 단계마다 ‘.md’ 파일을 생성하도록 해 봅시다.

- ❶ **1단계:** 이미지 인식 모델을 활용해 냉장고 사진에서 재료를 인식합니다.
- ❷ **2단계:** 텍스트 생성 모델을 활용해 인식한 식재료를 만들 수 있는 요리의 레시피를 생성합니다.
- ❸ **3단계:** 생성한 레시피를 저장합니다.

클로드 코드에서 PRD 생성하기

01 클로드 코드에 다음과 같이 입력합니다.

프롬프트

앞서 만든 OpenRouter API를 이용해서, 냉장고 사진에서 재료를 인식하고 레시피를 추천하는 웹 애플리케이션을 만들고 싶어. 다음과 같이 3단계로 나누어서 PRD를 만들어 줘.

1단계는 이미지를 입력받고 google/gemma-3-27b-it:free 모델을 사용해서 이미지를 인식해.

2단계는 1단계에서 얻은 정보와 deepseek/deepseek-chat-v3.1:free 모델을 사용해서 레시피를 생성해.
3단계는 사용자 프로필을 만들어서 레시피를 저장해.

각 단계를 PRD_step1.md, PRD_step2.md, PRD_step3.md로 나누어 저장해 줘.

> 앞서 만든 OpenRouter API를 이용해서, 냉장고 사진에서 재료를 인식하고 레시피를 추천하는 웹 애플리케이션을 만들고 싶어. 다음과 같이 3단계로 나누어서 PRD를 만들어 줘.

1단계는 이미지를 입력받고 google/gemma-3-27b-it:free 모델을 사용해서 이미지를 인식해.
2단계는 1단계에서 얻은 정보와 deepseek/deepseek-chat-v3.1:free 모델을 사용해서 레시피를 생성해.
3단계는 사용자 프로필을 만들어서 레시피를 저장해.

각 단계를 PRD_step1.md, PRD_step2.md, PRD_step3.md로 나누어 저장해 줘.■

02 다음은 교재에서 완성된 PRD의 요약입니다.

PRD_step1.md - 이미지 인식 핵심 기능 (5일)

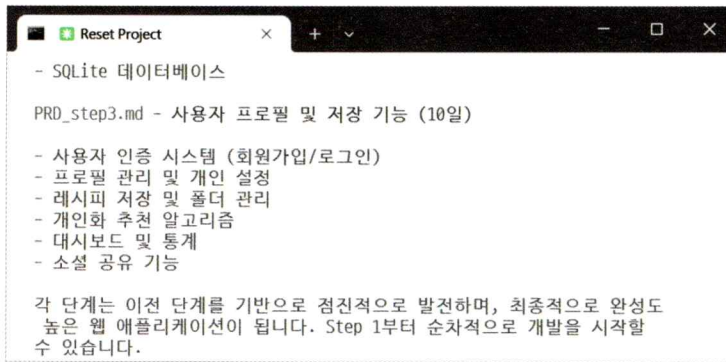
- Streamlit 기반 웹 인터페이스
- Gemma-3-27b-it 모델로 냉장고 재료 인식
- 이미지 업로드 및 처리
- 기본 UI 구축

PRD_step2.md - 레시피 생성 시스템 (7일)

- DeepSeek 모델로 레시피 생성
- 재료 편집 기능
- 필터링 및 검색
- 레시피 상세 보기
- SQLite 데이터베이스

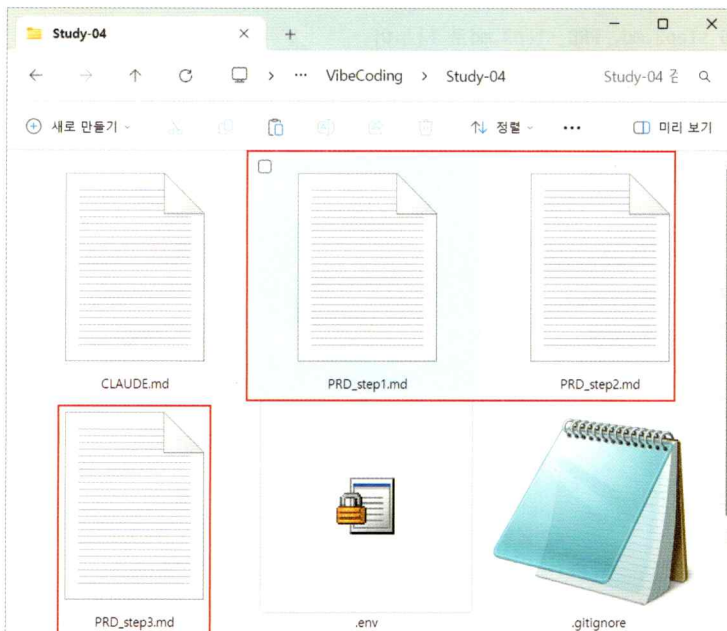
PRD_step3.md - 사용자 프로필 및 저장 기능 (10일)

- 사용자 인증 시스템 (회원가입/로그인)
- 프로필 관리 및 개인 설정
- 레시피 저장 및 폴더 관리
- 개인화 추천 알고리즘
- 대시보드 및 통계
- 소셜 공유 기능



화면과 책이 달라도 괜찮습니다.
자신만의 PRD로 진행하세요.

03 'Study-04' 폴더에 세 개의 단계별 PRD 문서가 작성된 것을 확인할 수 있습니다.



1단계: 냉장고 이미지에서 재료 인식하기

01 이제 작성된 PRD를 바탕으로 첫 번째 단계를 구현하겠습니다. 다음과 같이 프롬프트를 입력해 첫 번째 PRD를 실행하라고 명령합니다.

프롬프트

PRD_step1.md를 실행해.

```
> PRD_step1.md를 실행해.
>> accept edits on (shift+tab to cycle)
```


02 주어진 1단계 프롬프트에 따라 이미지 인식 기능이 구현되었습니다. 클로드 코드는 파이썬 웹 프레임워크 Streamlit을 사용하여 웹 인터페이스를 구축했습니다.

```
PRD Development
🚀 애플리케이션 실행
# 메인 애플리케이션 실행
streamlit run app.py

# 테스트 실행
python test_step1.py

웹 브라우저에서 http://localhost:8501로 접속하면 냉장고 재료 인식 시스템을 사용할 수 있습니다.

Step 1이 성공적으로 완료되었습니다. Step 2 (레시피 생성 시스템)를 진행하시겠습니까?

>

▶▶ accept edits on (shift+tab to cycle)
```

note 코드를 생성하면서 작업하는 모습을 터미널을 통해 확인할 수 있습니다.

03 이제 실제로 작동하는지 확인해 봐야 합니다. 아직 웹 서버가 실행되지 않은 상태라면, 클로드 코드에게 서버 실행까지 요청합니다.

프롬프트

메인 애플리케이션을 실행해서 1단계의 결과를 테스트해 줘.

> 메인 애플리케이션을 실행해서 Step 1의 결과를 테스트하게 해줘. █

▶▶ accept edits on (shift+tab to cycle)

+ 여기서 잠깐 파이썬 웹 프레임워크란?

웹 프레임워크 web framework란 웹 애플리케이션을 쉽게 만들 수 있도록 도와주는 도구 모음입니다. 일반적으로 웹 애플리케이션을 만들려면 HTML로 구조를, CSS로 디자인을, JavaScript로 동작을 분리하여 각각 프로그래밍해야 합니다. 하지만 파이썬 웹 프레임워크를 사용하면 하나의 파이썬 파일에서 이 모든 것을 처리할 수 있어 구조가 훨씬 단순합니다.

클로드 코드가 자주 선택하는 프레임워크는 Streamlit, Gradio, Flask 등입니다. 이 중 Streamlit은 특히 AI 모델을 시연하거나 데이터를 시각화할 때 자주 사용되며, 이미지 인식이나 텍스트 생성 같은 AI 기능을 웹에서 보여줄 때 유용합니다.

사용한 프레임워크에 따라 서버 실행 명령어도 달라집니다. 물론 클로드 코드에게 직접 실행을 요청하면 알맞은 명령어로 자동 실행해 줍니다.

- Streamlit을 사용한 경우, `streamlit run app.py`
- Flask를 사용한 경우, `python app.py`
- Gradio를 사용한 경우, `python gradio_app.py`

04 서버 실행이 완료되면 브라우저에서 접속 가능한 주소가 표시됩니다.

웹 인터페이스 접속

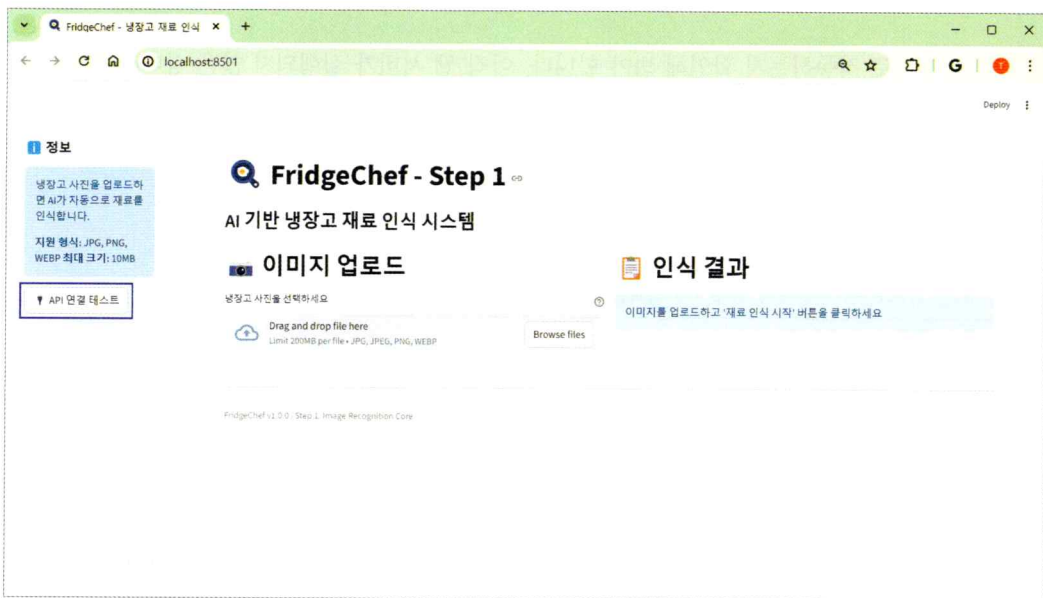
브라우저에서 <http://localhost:3000> 을 열어보세요!

사용 방법:

1. 이미지 업로드
 - 냉장고 사진 파일을 드래그 앤 드롭
 - "파일 선택" 버튼 클릭
 - "URL 입력" 버튼으로 이미지 URL 제공
2. AI 자동 분석
 - 업로드 즉시 AI가 자동으로 재료 분석
 - 재료명, 수량, 상태가 카드 형태로 표시
3. 재료 편집
 - ✏ 버튼으로 재료 수정
 - 🗑 버튼으로 재료 삭제

로컬호스트 포트 번호는 사용자마다 다를 수 있습니다.

05 로컬호스트 주소를 복사하여 브라우저에 붙여 넣으면 지금까지 구현한 결과를 브라우저에서 확인할 수 있습니다.



note 터미널상에서 **[Ctrl]** 키를 누른 채 주소를 클릭해도 웹 브라우저를 통해 실제 웹페이지를 열 수 있습니다.

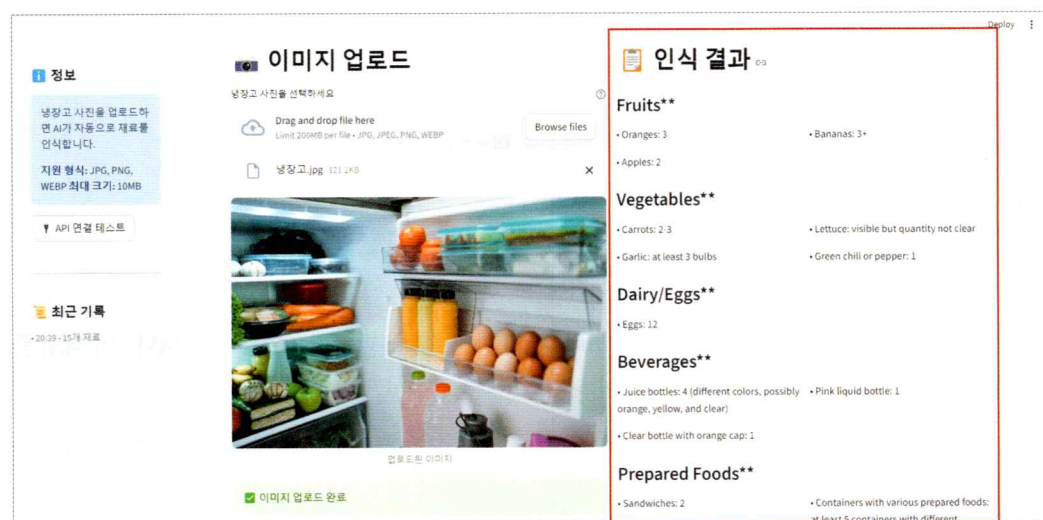
만들어진 애플리케이션의 세부 요소는 생성형 AI의 특성상 실행마다 조금씩 달라질 수 있습니다.



06 기능을 테스트해 보겠습니다. 깃허브 자료실(<https://github.com/taehojo/vibecoding>)의 '입력예제' 폴더에서 '냉장고.jpg' 파일을 미리 다운로드하세요. 그리고 나서 [Browse files] 버튼을 클릭하고 '냉장고.jpg' 파일을 선택한 뒤, [재료 인식 시작] 버튼을 클릭하세요.



07 잠시 후 AI 모델의 분석 결과가 화면에 표시됩니다.



note 아직 1단계이므로 디자인이 단순하고 영어로 출력될 수 있지만, API를 통해 실제 AI 모델이 이미지를 분석하고 있음을 확인할 수 있습니다.

2단계: 레시피 생성하기

01 이제 인식된 재료로 레시피를 생성하는 2단계를 구현하겠습니다. 다음과 같이 입력합니다.

프롬프트

이제 PRD_step2.md를 실행해 줘.

```
> 이제 PRD_step2.md를 실행해 줘. █
```

```
  ▶▶ accept edits on (shift+tab to cycle) · 1 background task
```

02 클로드 코드가 PRD_step2.md를 읽고 레시피 생성 기능을 추가하기 시작합니다. DeepSeek 모델을 연동하고, 재료 편집 기능과 레시피 표시 인터페이스를 구현하는 과정이 터미널에 표시됩니다. 작업이 완료되면 1단계와 마찬가지로 서버 실행을 요청합니다.

프롬프트

메인 애플리케이션을 실행해서 2단계 결과를 테스트하게 해 줘.

```
> 메인 애플리케이션을 실행해서 Step 2의 결과를 테스트하게 해줘. █
```

```
  ▶▶ accept edits on (shift+tab to cycle)
```

03 서버가 실행되면 터미널에 표시된 주소로 접속합니다.

지금 바로 브라우저에서 <http://localhost:8502> 접속하여 테스트하세요!

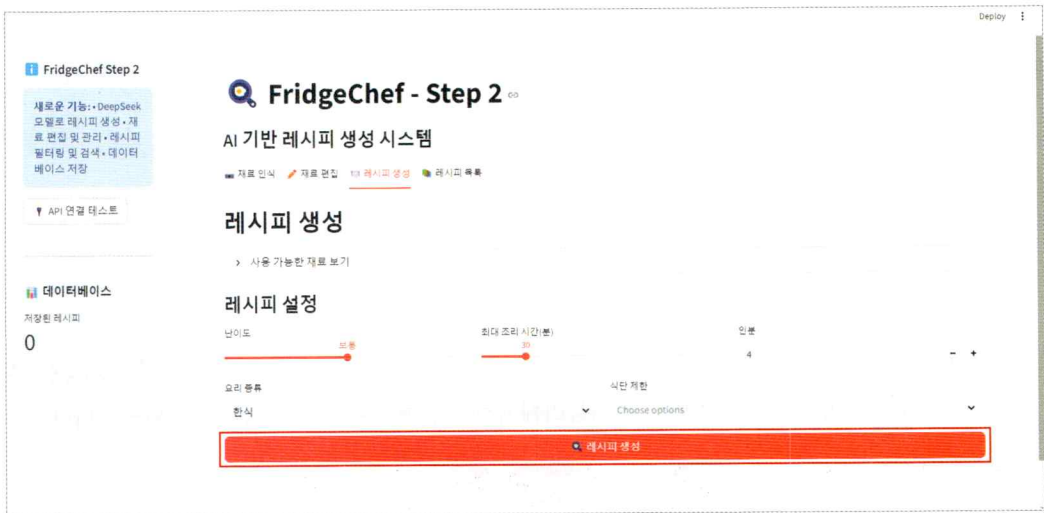
```
> █
```

```
  ▶▶ accept edits on (shift+tab to cycle) · 1 background task · ↵ to view
```

04 업그레이드된 애플리케이션에는 사이드바에 [레시피 생성] 메뉴가 추가되었습니다. 이 메뉴를 클릭하면 레시피 생성을 위한 다양한 옵션이 나타납니다.



05 인식된 재료를 수정하거나 추가할 수 있고, 요리 종류와 난이도를 선택할 수 있습니다. 설정을 마친 후 [레시피 생성] 버튼을 클릭합니다.



06 DeepSeek 모델이 선택한 재료와 조건에 맞는 레시피를 생성합니다. 각 레시피에는 요리명, 필요한 재료, 조리 순서, 예상 시간 등이 포함되어 있습니다. 이제 AI가 냉장고 재료를 인식하고 그에 맞는 레시피까지 추천하는 실용적인 애플리케이션이 되었습니다.



3단계: 사용자 프로필에 레시피 저장하기

01 마지막으로 사용자 프로필과 레시피 저장 기능을 추가하는 3단계를 구현하겠습니다. 다음과 같이 입력합니다.

프롬프트

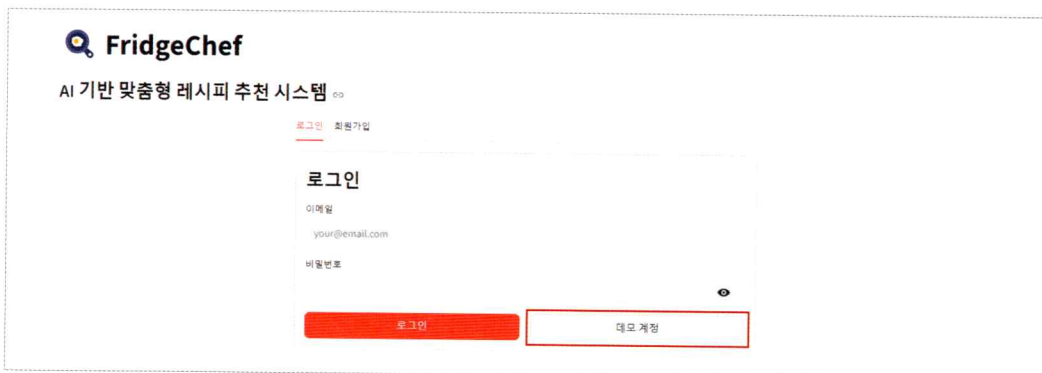
이제 PRD_step3.md를 실행해 줘.

```
> 이제 PRD_step3.md를 실행해 줘.
  >> accept edits on (shift+tab to cycle)  ·1 background task
```

02 클로드 코드가 'PRD_step3.md'를 읽고 사용자 프로필, 로그인 시스템, 레시피 저장 기능을 추가합니다. 3단계 작업이 완료되면 완성된 애플리케이션을 실행하여 새로운 기능들을 확인합니다.



03 애플리케이션을 실행하면 로그인 화면이 나타납니다. 아직 실제 데이터베이스와 연동되지 않은 프로토타입 ^{prototype}이므로, 화면에 표시된 테스트 계정 정보를 사용하여 로그인합니다. [데모 계정] 버튼이 있다면 클릭하여 데모용 임시 계정으로 테스트를 진행합니다.



note 프로토타입은 아이디어나 기능을 시험해 보기 위해 만든 초기 형태의 시제품으로, 디자인이 거칠거나 기능이 제한적일 수 있지만 완제품을 출시하기 전 개선 방향을 잡기 위해 필요한 초기 모델입니다.

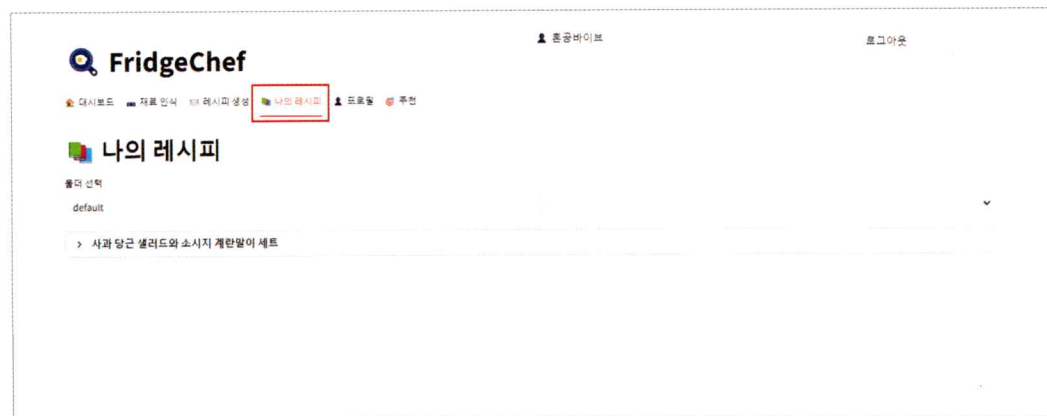
04 로그인에 성공하면 개인 대시보드가 표시됩니다. 저장한 레시피, 요리 횟수, 사용 통계 등을 한 눈에 볼 수 있습니다.



05 이미지 업로드를 통해 재료를 인식하고 레시피를 생성해 봅니다. 생성된 레시피 옆에 [저장] 버튼이 표시됩니다. 마음에 드는 레시피를 선택하여 저장해 보겠습니다.



06 [나의 레시피] 탭에 레시피가 저장된 것을 확인합니다.



07 대시보드로 돌아가면 통계가 업데이트된 것을 확인할 수 있습니다. 저장한 레시피 수가 증가하고, 자주 사용하는 재료 분석도 반영됩니다.



+ 여기서 잠깐 로컬 개발 환경의 이해와 한계

지금까지 만든 애플리케이션은 사용자의 컴퓨터, 즉 **로컬 환경** local environment에서만 작동합니다. 주소창에 입력하는 local host:8501의 'localhost'는 '내 컴퓨터'를 의미합니다. 즉, 다른 사람은 여러분이 만든 앱에 접속할 수 없습니다.

따라서 3단계에서 구현한 로그인 기능과 레시피 저장 기능도 임시적입니다. 브라우저를 닫거나 서버를 재시작하면 저장했던 데이터가 사라질 수 있으며, 여러 사용자가 동시에 접속하는 상황도 테스트할 수 없습니다.

하지만 이것은 한계가 아니라 개발 과정의 자연스러운 단계입니다. 실제 웹 주소로 누구나 접속할 수 있고, 데이터도 안전하게 보관할 수 있는 서비스를 개발하기 위해서는, 로컬에서 충분히 기능을 테스트하고 완성도를 높인 뒤, 실제 서비스로 전환하면 됩니다. 이를 위해서는 웹 호스팅 서비스에 애플리케이션을 배포하고, 클라우드 데이터베이스를 연결한 뒤, 도메인과 보안 인증서를 설정하는 과정이 필요합니다.

지금은 로컬에서의 개발과 테스트에 집중하고, 완성된 결과물을 서비스로 확장하는 단계를 차근차근 준비해 봅시다.

지금까지 '냉장고를 부탁해' 애플리케이션을 완성했습니다. 이미지 인식 AI를 사용해 사용자가 업로드한 냉장고 사진에서 식재료를 인식하고, 텍스트 생성 AI를 사용해 맞춤형 레시피를 추천 서비스를 구현했습니다. 클로드 코드와 AI 모델을 연결하는 과정에서 API를 활용하고, 애플리케이션이 적절히 작동하는지 테스트까지 마쳤습니다. 이제 다음 장에서는 AI 서버에이전트를 이용해 이 결과물을 점검하고 디자인을 수정하는 방법을 익히도록 하겠습니다.

마무리

▶ 2가지 키워드로 정리하는 핵심 포인트

- 이미지 인식, 텍스트 생성 등 목적에 맞게 여러 AI 모델을 활용하는 **다중 AI 모델 연동** 애플리케이션을 제작할 때는 단계별 계획에 따라 점진적으로 완성도를 높여야 합니다.
- **웹 프레임워크**는 웹 애플리케이션을 쉽게 만들 수 있도록 도와주는 도구 모음입니다.

▶ 확인 문제

1. PRD를 3단계로 분리하여 개발할 때, 2단계에서 DeepSeek 모델 API 호출이 실패하면 다음 중 점검이 필요한 항목을 모두 고르세요.
 - ① .env 파일의 OPENROUTER_API_KEY 존재 여부
 - ② DeepSeek 모델명의 정확성
 - ③ 1단계에서 Streamlit 서버가 이미 8501 포트를 사용 중인지 여부
 - ④ .gitignore 파일의 존재 여부
2. Streamlit과 전통적인 웹 개발(HTML/CSS/JS)의 차이점으로 틀린 것을 고르세요.
 - ① Streamlit은 파이썬 코드만으로 웹 앱을 만들 수 있다.
 - ② Streamlit은 데이터 시각화와 ML 모델 시연에 특화되어 있다.
 - ③ Streamlit은 반드시 프론트엔드와 백엔드를 분리해야 한다.
 - ④ Streamlit은 'streamlit run' 명령어로 실행 가능하다.
3. 프로젝트에서 API 키가 깃허브에 업로드되는 것을 방지하는 구체적인 방법을 서술하세요.